CODE SUBMISSION AND STYLE GUIDELINES

CMPUT 274/275:
Introduction to Tangible Computing

2019-2020

Department of Computing Science
University of Alberta

Table of Contents

Section 1

Code Submission Guidelines

- Learn the academic integrity and general file submission guidelines that are expected of you in CMPUT 274.
- Know that you must always list all sources used and anyone you collaborated with on any project. The penalties for plagiarism are severe, and should be avoided.

1.1 Academic Integrity

Any submission in any class should follow these rules:

1.1.1 Label Your Submissions

Include a File Header. Always label your submissions (any code files) with your name, and student ID. **Every file you submit must have a file header in comment form** that describes the following information:

- your name and student ID
- the course name, term, and assignment name
- you may also want to include a brief description of the purpose of this file if you have multiple files. However, this is not required as this information will usually be included in your README.

Here is an example of a complete file header for a Python exercise. You do not need to copy the style of this file header (only the information is important, how you structure it is up to you):

```
# ------------------------------------------------
#   Name: John Smith
#   ID: 1234567
#   CMPUT 274, Fall 2019
#
#   Exercise 1: Hello World
# ------------------------------------------------
```

Joint Submissions. If the submission is a joint submission as part of a group or partnership, you must include **all students' names**. Only the student submitting the assignment should include their student ID. Do not share your student IDs! **Only one person per group should submit** the joint assignment.

1.2 Levels of Collaboration

Note that different assignments in this course allow different levels of collaboration. Weekly exercises and morning problems are a "Solo Effort" and **must be solved individually, without consultation with anyone besides the instructors and the TAs**. However, **some** major assignments and the final project are done under the "Full Collaboration" model. These assignments must be completed in pairs: each student must pair up with precisely one other student. Students may not complete these alone or in a team with more than two students. **In CMPUT 274, you will be able to collaborate with exactly one other student on Assignment 2. Assignment 1 (Huffman Coding) is to completed individually under the "Solo Effort" model of collaboration.**

The collaboration models are outlined here: https://www.ualberta.ca/computing-science/links-and-resources/policy-information/department-course-policies.

It is your responsibility to make sure you understand the difference between the levels of collaboration allowed on the various assignments in the course. We will take issues with academic integrity seriously in this course, so be careful.

1.2.1 Plagiarism Concerns

Important:

- Always list anyone you collaborated with and the nature of the collaboration.
- Always describe any code you copied from any source, and list its source.

1.3 File Submission Guidelines

Any files submitted on eClass must follow the following conventions. Failing to adhere to these instructions may result in a mark deduction.

1.3.1 Weekly Exercises, Assignments, Final Project

All code and other supporting files, including the README, must be submitted in a compressed archive:

- The file can be one of **.zip or .tar.gz only**.
- This must be done even if the assignment contains only a single file.

You must include a README in all submissions **unless otherwise specified**. See the next chapter on writing READMEs for more specific guidelines.

1.3.2 Morning Problems

For morning problems, simply submit the source code as a single **uncompressed** file (.py or .cpp). You may only submit **one file** for the morning problems.

1.4 Style Guidelines

All submissions should follow any stylistic guidelines discussed in class or demonstrated in provided code examples. This includes:

- Proper indentation that makes clear what block (if/else/while/for) a statement belongs to. This is especially important for C++/Arduino code.
- Variable names that are descriptive of their purpose.
- Comments that describe any tricky part of code or give high-level structure.
- Anything that improves readability and understanding.

We have some more exhaustive suggestions for programming style that you should generally follow in future sections, as well as some explanations for why you might want to do certain things. **We will be looking at style when marking, so keep this in mind when writing your code.**

Section 2

README Guidelines

Learning Outcomes:

- Learn the purpose of a README and how to write one effectively.

2.1 What is a README?

A README file is just a plain text file that contains clear, concise information for the user about a software program, utility, or game. You will create your README files to commuicate with a marking TA and provide running instructions, explanations about your process, wiring instructions for the Arduino, and detail of any known bugs in your code.

You can create a plain text file using any basic text editor. For example, you can write a README in Windows using Notepad, in Linux using Gedit, or on a Mac using Textedit. On the VM, you can also use the text editors used in class, Sublime Text or Atom.

2.2 Filename Specifications

The full filename of your README must be **README only with no extension**. If you first create something like README.txt, rename it to README.

2.3 Required Components

Your README should include a **header** with the following information:

- your name and student ID at the top
- the course name, term, and the name of the assignment or exercise
- details of collaboration or copied code as noted in the Academic Integrity section above (if applicable).

In addition to this, your README should include **all information that someone would need to use your code. This includes, but is not limited to:**

- running instructions that detail how your code is run (i.e., load into the Arduino IDE and upload from there)
- description of any assumptions you made in your implementation (e.g., the user cannot enter numbers bigger than 100)

- a list of any functionality you included that is outside the exercise/assignment specification
- For Arduino-based submissions only:
  - a list of all required components needed to run your program (the Arduino MEGA itself must be included in this list!)
  - wiring instructions explaining how your Arduino must be wired to run your program
- For submissions including a custom `Makefile` only (see section 3):
  - Include a list of all targets in your `Makefile`, along with a single-line explanation of each target. For example:

```
Makefile Targets:
    - make (project): Builds the project.
    - make clean: Removes all object files and executables.
    - make help: Prints a list of targets and the purpose of each.
```

The README is also how you can communicate with your grading TA. You may also want to mention problems you ran into with your submission. For example, you may want to detail the following information as it will help your grading TA:

- if your code is incomplete
- if your code has known bugs or does not work properly
- if you are aware that you implemented something in a strange way but could not reach the appropriate solution

In general: your README should include any information that can make your TA's life easier. Pointing out problem areas makes it easier for them to assess your code AND makes it easier for them to give you part marks. **Make sure to be clear and concise, and avoid rambling**.

Note: Including a README file is a standard for developers. This is where you can tell users how to use your code. You want to make it easy for others to use your code, so you want to be clear and include all relevant information. READMEs in this class should help get you in the habit/mindset of always including a README file. Thus it may seem silly to include your wiring instructions for an exercise, when they were provided in the exercise description, but it is information you'd want to give to anyone using your code so it should be added to your README.

2.4 Example README

On the next page is an example of a README that would receive full marks, based on a README submitted for an Arduino/C++ exercise in a previous year. Any text within `/*` and `*/` markers is a comment and not part of the README itself. Note that this is not a required format but following this will ensure that you cover all your bases:

```
Name: STUDENT NAME
ID #: 1234567
CMPUT 274 Fa17

Exercise 4: String Functions
```

/* Included Files is not strictly necessary as per the guidelines above, but helpful for clarity. It may also be helpful to include a brief description of what is included in each file. */

```
Included Files:
    * exercise4.cpp
    * string_processing.cpp
    * string_processing.h
    * README
    * Makefile

Accessories:
    * Arduino Mega Board (AMG)

Wiring instructions: None needed
```

/* Running instructions must contain all the information necessary for someone to run your code. Don't forget to specify that the user must open the Serial monitor to view the output. */

```
Running Instructions:
    1. Connect the Arduino to the PC using an A-B style USB cable. Ensure
   that the Arduino is using the proper serial port (/dev/ttyACM0 or -ACM1)
   .
    2. In the directory containing the files exercise4.cpp,
   string_processing.cpp, string_processing.h, and a Makefile, use the
   command "make upload && serial-mon" to upload the code to the Arduino
   and view the serial monitor.
```

/* State any assumptions, descriptions, or comments for your grading TA. */

```
Notes and Assumptions:

    The file exercise4.cpp contains a function called HelloWorld which
   demonstrates the getStringLength() and printBackward() functions using
   the string, "Hello, World!". Then, the program runs some test code,
   including edge cases.

    The program assumes that the string taken as an argument will have an
   length that is an integer. The string thus must be between 0 and 32,767
   characters (assuming it is stored in 16 bits) or the program will
   function incorrectly because of overflow.
```

2.5 Example Wiring Instructions

The wiring instructions can be as follows (modify the instructions to reflect your specific setup; the setup below is valid for the standard single LED setup that is used in the Blink lab in the Arduino Intro Labs):

```
Accessories:
    * 560 Ohm (Grn Blu Brown) resistor
    * LED
    * Arduino Mega Board (AMG)

Wiring instructions:
    Arduino Pin 13 <--> Resistor <--> Longer LED lead |LED| Shorter LED
   lead <---> Arduino GND
```

Section 3

Makefile Guidelines

In CMPUT 275, you will be eventually expected to submit a `Makefile` to compile the C++ assignments and exercises that are not Arduino-based. You must create a specific `Makefile` to compile each project. We will specify when a `Makefile` is required in the assignment/exercise description.

The basics of `Makefiles` will be covered in class, this section is only relevant after this.

Here are a few general rules to follow:

- Include all targets specified in the exercise description, with precisely the same names and expected behaviour.
- No matter what is specified, you must always include the target `clean`, which removes all object (.o) files and executables. Note that you should also run this command before submitting to ensure that your directory is clean.
- Use `Makefile` variables to specify compiling and linking flags. This way, if the term needs to be changed, the modification need only be made in one place.
- Decompose complex targets into multiple recursive dependencies. For example, it is good practice to create separate targets to compile each object file. Then, use another target to link all relevant objects together and specify these objects as dependencies. Similarly, each `.o` target should list the source code files and header files the object depends on that are included in your submission (eg. any files you develop or use from class, but not files from the standard library).

In addition to the above guidelines, all of the following are acceptable modifications to any submitted `Makefile`:

- Organizing source code and object files into subdirectories (`src` and `obj`, respectively). This keeps your source code separate from object files and executables, which improves project organization. We encourage this as best practice, but it is not required. If you choose to create these subdirectories, simply ensure that:
  - the executable file is generated into the main directory (not into `src` or `obj`).
  - you include both subdirectories in the compressed archive you submit.
- Adding your own additional targets (as long as these do not interfere with the intended targets). Make sure that any such targets are below the main target so it is not built by `make`.
- Suppressing the output of a target using `@` (ie, to avoid printing the output of `rm`).

- Printing helpful messages using `echo`. Please limit this to one line of output per target and document any messages in your README. An example acceptable message is "Executable `prog` successfully generated.". Excessive or unhelpful messages may result in deductions at the grader's discretion.

Documentation:

You are not required to put descriptive comments inside your `Makefile`. However, please include the standard file header you are accustomed to placing at the top of every file. Additionally, you must insert a list of all your `Makefile` targets in your README along with a brief one-line explanation of the purpose of each one. See Section 2.3 for more detail.

Section 4

Programming Style Suggestions

Learning Outcomes:

- Learn the style and good practice guidelines that are expected of you in CMPUT 274.
- Understand the reasoning behind clear, consistent style and the importance of comments.

4.1 Why Style is Important

Any code that you submit in CMPUT 274/275 will be graded for style and documentation (unless it is part of a morning problem solution). As you progress through the course, the requirements for good style will become more strict. Somthing that gets away with a warning on the first weekly exercise will likely result in a grade deduction later on.

Therefore, it is very important to keep these programming style suggestions in mind. Additionally, pay attention to what your grading TA tells you in the feedback comments you will receive for most assignments. These comments will give you a good idea about what specifically you may need to change about your coding style.

Here are a few general goals of good style. These are the most important things to keep in mind:

- **Clearly organize your code. Try to make it modular.** You can modularize your code by using functions or loops to avoid redundant code. A clear organization is easy to follow and work with.
- **Always confirm that your code compiles on the VM and runs without errors before you submit! It is your responsibility to check that your code works on the VM.** It is not your grading TA's job to fix minor bugs in your code until it compiles/runs successfully. If there are many errors or they are difficult to fix, you may even get a grade of 0 on the assignment. Make sure to run your code right before you submit, and if you are not using the VM for development, make sure to test it on the VM as this is the only officially supported environment.
- **Document your code.** Include file headers and comments that describe what your code is doing. Try to explain WHY you are doing things rather than describing WHAT you are doing, especially for simple pieces of code.
- **Do everything you can to make your grading TA's life easier.** Make your program easy to read and follow. Include any important notes about your implementation in your README. If your grading TA has to sift through messy or difficult code, you will lose marks even if your submission is functionally correct.

4.2 Documenting Your Code

Please comment your code. Tell us at a high level exactly what your code is doing. Don't tell us what every single line of code does, but make sure you comment anything that might be confusing. Your comments should describe what the overall goal of your code is, and what the algorithm to achieve that goal is doing. This is excessive:

```
/* Add 1 to number */
number = number + 1;
```

We already know that, the comment just repeats what that line of code already says! The code does have a voice of its own. However, you should try to comment what your code is doing - but you want to be a little less verbose than that. You want to essentially explain the main steps like you would if you were explaining how to do a math problem to somebody, and you want to make note of anything that can go wrong and anything that might be a little strange.

Almost all of the code you write should have comments of some sort! Here are some general guidelines:

- Include a file header in every file you write (see the section "Label Your Submissions" from the first chapter for more detail).
- Include a function comment at the start of every function describing its purpose.
- Try to describe WHY you wrote your code the way you did, rather than WHAT it does.
- Include a comment at the start of each section of code, such as the start of a loop, an if statement, or a variable describing its purpose. For example, this double for loop structure, which prints all the primes from 2 to 100, is well-documented.

```
# print all primes from 2 to 100 (inclusive) using trial division
for i in range(2, 101):
    # we assume each number is prime by default
    prime = True
    for j in range(2, i):
        # if i is divisible by any number from 2 to i-1,
        # then it is not a prime.
        if (i%j == 0):
            prime = False

    # nothing is printed if i is not a prime
    if prime == True:
        print(i, "is prime")

print("Good bye!")
```

- Make sure you document anything that could be confusing in your code. Explain anything unusual or unconventional.

4.3 Function Headers

You must have a comment header for your each of your functions, explaining what they do, what arguments they take, and what they return. It's also a very good idea to make note of what global

variables and other external factors that might affect the function's return values, or that the function might modify.

Here is an example of a good function comment structure in C++ for a simple function that computes speed given distance and time:

```
/*
    Description: Computes the speed of a cyclist during a race.

    Arguments:
        distance (float): the distance covered in km during the race
        time (float): the time taken to traverse distance in hours

    Returns:
        speed (float): the average racing speed of the cyclist.
*/
float compute_speed(float distance, float time) {
    cout << "distance covered in km: " << distance << endl;
    cout << "time taken in h: " << time << endl;
    float speed = distance / time;
    return speed;
}
```

And the same function in Python:

```
def compute_speed(distance, time):
    """ Computes the speed of a cyclist during a race.

    Arguments:
        distance (float): the distance covered in km during the race
        time (float): the time taken to traverse distance in hours

    Returns:
        speed (float): the average racing speed of the cyclist.
    """
    print("distance covered in km:", distance)
    print("time taken in h:", time)
    speed = distance / time
    return speed
```

Note that it is standard to include the function header outside the function in C++ and inside of it in Python. When a multi-line function header is included in this location in Python, it is called a **docstring**, and can also be used to store test cases. Also note that classes and structs should have comment headers in the same style as well.

Although this particular comment structure is not required, it is an extremely clear way to document your code. You must implement something like this for functions that you submit in CMPUT 274 and 275.

4.4 Variables

Here are a few general guidelines for naming variables:

- Try to avoide single-character variable names wherever possible.
- Use descriptive variable names that reflect what you want to store in them.
- Avoid using global variables where local ones make more sense.
- Do not use builtin keywords or function names (such as `list` in Python) as variable names. Using a builtin keyword or function name as a variable overwrites and replaces the original meaning. For example, in Python, setting `print=5` will override the builtin `print` function, and you will no longer be able to use it correctly. Generally, if a text editor highlights your variable name in a different colour, you should probably change it.

4.4.1 Single Character Variable Names

You should usually avoid using single characters for variable names when you could have a more descriptive name.

With a single character variable name you cannot easily tell what it is supposed to represent (in most cases). For instance, a variable named 't' could refer to 'temp', 'testcase', 'time', or something else entirely. Even if you think you can remember which it is, imagine going back to the same piece of code in a month, or a year, or longer. Alternately, imagine trying to understand someone else's code when the variable names are all single letters. The variable 'time' is better than 't' because it tells you more clearly what the variable is supposed to represent.

Knowing what a variable is for allows you to know what to store in the variable and what calculations you can perform with its contents. If you can give a variable a name that tells you what it represents, then you should.

The two exceptions to this are:

- When the variable in question is one letter by convention. For example, x and y are default variables used to represent points on the Cartesian plane, and i is commonly used as an iterator.
- When a variable is general (or abstract) enough that you cannot make the name more specific. For example, mathematical equations often use one-letter variables because they do not represent any specific type or value. You will not usually need this level of abstraction, especially when you begin to program.

4.4.2 Use Descriptive Variable Names

Similarly, you should always try to use descriptive variable names. Names like "variable" are just as useless as a name like "v". I still don't know what "variable" might represent. Make your variable names reflect what you want to store in them. It's just like how you should store cumin instead of rat poison in the jar labeled "`cumin_jar`". You don't want to accidentally poison your food (or have

somebody else, unaware of the rat poison, cooking in your kitchen poison their food). Similarly, you don't want to accidentally use the wrong values in your program. If you do that it won't work! Often other people have to read and modify / use your code, so it needs to be readable to make sure they don't accidentally use the wrong variables as well!

To put it simply, if your variables are not named descriptively then you, and the people reading / using your code, are much more likely to get confused and make mistakes - which will make your program not work as intended!

4.4.3 Avoid Global Variables

A global variable is a variable defined outside any function of your program, and is accessible by all functions.

Avoid using global variables when local variables would make more sense. Since any function can read / write to a global variable, it's difficult to keep track of what value is stored within the global. If a function is called you have to make sure it does not change the global variable in an unexpected way which might introduce bugs into your program, and that's really tedious!

Global variables can also make your code more difficult to reason about. Consider the program:

```
#include <Arduino.h>

/* Evil global variable! */
int global = 0;

int function(int number)
{
    /* Increment our global variable */
    ++global;

    return global * number;
}

void setup()
{
    Serial.begin(9600);

    /* Print the result of the function a few times to show global's
   effects */
    Serial.println(function(4));
    Serial.println(function(4));
    Serial.println(function(4));

    /* We can even change the result of the function by changing global! */
    global = 0;

    Serial.println(function(4));

    /* Hmmmm, what's global now? I just set it to 0, right!? */
```

```
    Serial.print("Global: ");
    Serial.println(global);
}

void loop() {}
```

You might expect calling function(4) to return the same result every time, after all it's the same piece of code, right? Well, running this program you should see

```
4
8
12
4
Global: 1
```

Because you increment global each time the function is called, and the result of the function relies upon that external piece of state (the return value is multiplied by it), we don't get the same return value each time! We multiply by a slightly larger global variable each time we call the function.

In more complicated functions this can be a nightmare when debugging, because when you look at your function it might appear like it's doing the right thing, but if a value of some global that it relies upon is incorrect the return value of the function will also be incorrect (even if there is nothing wrong with that function!)

Also note that it can cause a bit more confusion too! For instance in setup() we very clearly set global to 0, so we might think that it still has that value later on. But, global is in fact changed, it's just that the change is hidden away in the function call!

It is often very desirable to have what is called a pure function, which is basically just a fancy way of saying "a function that will always give you the same return values for the same inputs, and does not change any variables which can be seen by anything else!" Since pure functions don't cause any side effects they are a bit easier to debug because you can trust that they haven't messed with any other part of your program, and no other part of your program can mess with it. A pure function is isolated from the rest of your code, so any mistakes made in the pure function can't break anything else, and mistakes made elsewhere can't break your pure function! It makes them easy to test too!

NOTE: There are some situations where global variables are desirable, so this is not a hard and fast rule. For example, constants that are set at the beginning of the program and never modified are often global so that they can be accessed easily by all functions. For this reason, Arduino code often has many global variables, and they are not always a bad thing.

4.4.4 Avoid Using Reserved Keywords

It is important, especially in Python, to avoid using a **reserved keyword, builtin function name, or builtin type** as a variable name.

Unfortunately, Python is different from C++. If you use a reserved keyword, `true` as a variable name in C++ like this:

```
#include <iostream>
using namespace std;

int main() {
    // true is a reserved keyword
    int true = 5;
    return 0;

}
```

You will see the following error message:

```
true.cpp: In function int main():
true.cpp:5:9: error: expected unqualified-id before true
     int true = 5;
         ^
```

However, the Python interpreter will not raise an execption or print an error message if you do something like this, because it assumes you did this intentionally. This can have unintended consequences.

For example, in Python 3, the word "print" represents the builtin function `print`. Do not name your variable print like this:

```
>>> print = "hello"
```

As you can see, the word `print` is highlighted in pink. Using this name will override the builtin functionality of the word "print" and replace it with the string "hello". If you do this and then you try to print something, you will get an error:

```
>>> print(5)
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'str' object is not callable
```

Generally, if your text editor highlights your variable name in a strange colour, you should probably change to a different one. Other common mistakes in Python are to use the builtin types `list` or `str` as variable names. Instead, use "`my_list`" or "`my_str`", or (ideally) create a more descriptive variable name.

4.5 Avoid Hardcoding

Often, your goal will be to make your code **versatile** and **reusable**. Hardcoding is the practice of writing inflexible code with special "magic" values that are meaningful to the original coder when written, but as time passes may be difficult to interpret. Hardcoding values makes code less readable and may hamper the development of your code.

For example, imagine that you want to make a program that draws an animated tree that sways slowly from side to side. You decide that every leaf should have a constant width of 6. Throughout your program, you type this number directly wherever you need to draw or modify a leaf. When

you are almost done, you change your mind and want all the leaves to have a width of 10 instead. Unfortunately, you now have to track down all the locations where you used this number and change it, without making a single mistake! Using a **global constant** called `LEAF_WIDTH` would have solved this problem, because the leaf widths could be changed quickly and assuredly by changing `LEAF_WIDTH = 6` to `LEAF_WIDTH = 10`.

4.5.1 Constants and Magic Numbers

As opposed to global variables, global constants are actually a good thing! You want to avoid "magic numbers" in your code whenever possible. A magic number is essentially any literal numerical value put in your code, that could instead have a name assigned to it. For example,

```
for (size_t name_index = 0; name_index < 20; ++name_index) {
    char *name = name_array[name_index];
    Serial.println(name);
}
```

Is less readable than if you had defined somewhere in your code a constant like,

```
const size_t AMOUNT_OF_NAMES = 20;
```

and then had

```
for (size_t name_index = 0; name_index < AMOUNT_OF_NAMES; ++name_index) {
    char *name = name_array[name_index];
    Serial.println(name);
}
```

Because you don't know what "20" represents otherwise. An additional advantage of doing this is that if you have a bunch of loops like this in your code, and you need to add another name you can actually just change your constant in one place, instead of scouring your code for all of the magic numbers that you have to change, which is very time consuming.

4.6 Code Structure

The structure of your code is an important aspect of style. Here are a few suggestions to keep in mind as you code:

- **Nicely space your code** to group similar statements (such as the lines in a function or loop) together. Separate different components with blank lines to make them easy to distinguish.
- **Use proper indentation** that is consistent and helps make your program more readable. It is especially important to do this in C++, because unlike Python, C++ is not structured using indentation. If you are not careful, your code can become very messy, very quickly.
- **Make your code modular** wherever you can by using loops and functions. This means keeping repeated code to a minimum, and clearly organizing your code into sections. For example, if you have to calculate the average of a set of numbers in more than one place in your program, make a single function you can call to accomplish the task. Avoid needlessly repeating large blocks of code with only small changes!

- **Keep your functions relatively small** to improve readability. If you have a 100 line function, it can be very difficult to understand and debug. A good rule of thumb is to try to keep your functions to 20 lines or less.
- Each function should be **responsible for only one task**. For example, a program that reads a text from a file, analyzes the text, and prints a summary should not be implemented all in one function! Divide this into at least three functions: one to read in the text, another to do the analysis, and a third to print the summary. You may even need multiple functions to complete the analysis if it is done in multiple parts.
- Above all, be consistent with the style decisions you make.

4.7 Indentation

Indentation is the amount of whitespace at the beginning of each line of a program. It is used to convey the program's structure. Languages like C or C++ do not require indentation, and proper, consistent indentation is done for style and readability. Other languages, including Python, have precise indentation constraints. Here are a few general guidelines about indentation style:

4.7.1 Use Spaces, Not Tabs

Spaces should be used for indentation instead of actual tabs. The rationale for this is that tabs may display differently on different computers and editors, so an amount of indentation that is acceptable for you may not appear nicely on somebody else's computer. With a fixed-width font, which everybody should use, spaces will always have the same amount of indentation relative to the text.

Note that the editor that we will be using (Sublime) is by default configured so that the tab key does indentation with spaces on the VM. You can configure other editors (Emacs, Vim, Atom) to do the same thing.

4.7.2 Avoid too Little or too Much Indentation

Indentation should be a minimum of four (4) spaces. Four spaces is a good amount of indentation, but you can probably go as high as eight (8) spaces of indentation if you prefer more. The rationale for this is that indentation is used to separate code inside of different blocks, and any less indentation is more difficult to notice at a glance. Too much indentation can also make things unreadable because there will be too much blank space on any given line.

```
int stuff = 0;
stuff++;

if (1 == stuff) {
    /* 4 spaces - Easier to see this is inside the if */
    stuff++;
}
```

```
if (2 == stuff) {
  /* 2 spaces - Harder to tell */
  stuff++;
}

if (3 == stuff) {
/* None - Things are getting crowded now, and this is impossible */
stuff++;
}
```

This effect is made much worse if the blocks for the if statements are much larger, or there is a lot of code around an if statement. If it's indented by a large enough amount you can tell right away what a statement is nested in. You can tell if it's in a loop, or in an if statement inside of the loop very easily.

4.7.3 Inconsistent Indentation

Of the problems with indentation, this one is probably the worst. You must use a consistent amount of indentation within a file. If you are indenting with 4 spaces you always indent by 4 spaces for each level of indentation. Changing the amount of indentation randomly makes code very hard to read. Here is a bad example:

```
int stuff = 0;

     stuff++;
   stuff += 2;
            stuff = stuff * 7;

    while (stuff) {
        stuff = stuff + 3;

if (stuff) {
    stuff = 8;
}
     else {
   stuff = 9;
          }
    }
```

It's very hard to tell what goes where here, it's certainly much more difficult to read than a more properly indented piece of code which does the same thing:

```
int stuff = 0;

stuff++;
stuff += 2;
stuff = stuff * 7;

while (stuff) {
    stuff = stuff + 3;
```

```
    if (stuff) {
        stuff = 8;
    }
    else {
        stuff = 9;
    }
}
```

In the first case it's not really clear what is inside of the while loop at first, but it is very obvious in the second case. The first example actually leads to a lot of mistakes! People get confused thinking that the if and the else statements are not inside the while loop, so their code doesn't behave how they expect it to.

Keep in mind that Python is particularly picky about this kind of stuff - your code likely won't work if you have inconsistent indentation! Therefore, it's a good idea to get in the habit of doing this consistently to avoid unnecessary errors.

4.8 The Pep8 Style Guidelines

This course will follow the coding conventions of the official PEP 8 Style Guidelines. Take the time to skim through it now and use it as a reference when you have questions about Python style conventions. In particular, here are a few highlights that have not already been mentioned in this document:

- The maximum line length is 79 characters. This means that all lines in your program should be limited to 79 characters at most.
- Use two blank lines to separate top-level functions and class definitions, and use blank lines in functions (sparingly) to indicate logical sections.

Although some of the PEP 8 recommendations are very restrictive, if you follow these guidelines exactly, you will never lose marks for style.

4.9 The VM's Style Checker

The VM has a style checker that can be invoked on a program you have written. It works for both Python and C++ and follows the PEP 8 style conventions.

It can be invoked as follows:

```
style programname.ext
```

where `programname` is the name of the program you wish to check and `ext` is the extension (either .py or .cpp).

It is not necessary for your program to pass the style checker in order to receive full marks on an assignment. However, you can be fairly certain that if your program passes the style checker's rather restrictive requirements, you probably do not have any style issues.

Note that the style checker does not correct style issues. It is up to you to make the necessary changes according to its output. It is also not perfect. For example, it will return no errors for a correctly indented program with no comments, even though you are required to include documentation in your code.

Also note that although the checker works for C++ programs, it cannot check for problems with indentation. Instead, it can only return a warning indicating that you may have indentation issues in your program.

Make sure that if you run your code through the style checker and it passes, you also visually inspect your code and run it to make sure you do not have additional errors that the style checker is not designed to catch.